

Self-Test framework for Self- Adaptive Software Architecture

Y. Mohana Roopa
Associate Professor, Dept.of CSE
Institute of Aeronautical Engineering,
Hyderabad, INDIA.
ymohanaroopa@gmail.com

Dr. M. Ramesh Babu
Professor, Dept.of ECE
Institute of Aeronautical Engineering
Hyderabad, INDIA
ramesh01077@gmail.com

Abstract— as researchers and contributors of the IT enterprise move in the direction of a vision of computing structures that manage themselves, it's far imperative to research methods to dynamically validate these systems to keep away from the high cost of system failures. Although research continues to advance in lots of areas of self adaptive systems, there's a loss of development within the area of testing components of systems at runtime. Self-adaptive features in systems dynamically invoke changes to the structure and behavior of components which could already be operating in an unpredictable surroundings; similarly emphasizing the need for runtime testing. In this paper we propose a framework that dynamically validates modifications in self-adaptive systems. Our body-work extends the present day shape of self-adaptive systems to consist of self-testing as an implicit characteristic. We validate our framework by way of creating a prototype of an autonomic container that incorporates the capability to self-test.

Keywords— *self adaptive systems, component testing, Validation, safe adaptation.*

I. INTRODUCTION

As technology-oriented business organizations continue to grow in size and complexity, there's a need to shift the load of help tasks which include configuration and fault management from humans to new generation [5]. IBM's autonomic computing model is a movement toward self-managing structures that automatically configure, heal and optimize themselves [8],[10]. This model has efficaciously attracted participants from both the IT industry and the research community [9], and has brought about the improvement of plenty research projects based on autonomic computing.

Although research continues to advance many areas of self adaptive systems, there is a lack of development in the area of components of self adaptive systems at run time. Dynamic changes and reconfigurations which are resulting from self adaptation are applied and accepted with focus on validating them against the system requirements.

The goals are set as high a level policy which include changes to system components and gives a

pay for system administrators to check if the component is operating within the bounds of some desired behavior.

However, such a technique is not sufficient to decide if the system behavior nonetheless conforms to the general functional and nonfunctional requirements after such modifications had been carried out. The principal contribution of this paper is the development of a methodology that supports automatic runtime validation of modifications resulting from self-testing components in self-adaptive systems. Our method:

- 1 Automatically doing self-testing on change re-quests in self-adaptive systems at run-time, and evaluates test at adequacy through the usage of a validation policy file.
- 2 Extends the architecture of self-adaptive systems by using applying ideas of Orchestrating and Touchpoint Autonomic Managers [8] to testing activities.
- 3 Provides two techniques for validation based on system overhead cost and feasibility of use safe adaptation with validation, and replication with validation.
- 4 Integrates testing into the current workflow of self adaptive managers through communications to new managers designed for testing.

This paper is organized as follows: the next section contains related work on autonomic computing and self adaptive systems; section 3 briefly describes approaches to testing evolving and adaptive systems. Section 4 provides an overview of our testing methodology. Section 5 presents the architecture of the proposed self-testing framework. Section 6 provides Experimental results on the prototype used to validate the proposed testing methodology. Section 7 presents concluding remarks of the work.

II. RELATED WORK

The fact that there is clearly a need for self adaptive systems to perform self-testing [10],[12], few

researchers have addressed this issue.

Several researchers have investigated the notation of self-testing software [1], [2], [11], [14], [15]. The work by Le Traon et al. [14] is most carefully related to this work; it describes a realistic technique to develop self-testable components that link layout to the testing of classes. Components are self-testable by means of including test sequences and test oracles of their implementation. For this technique to be practical at the system level, structural test dependencies between self-testable components have to be taken into consideration at the system architecture level. The concept of self-testable components strongly supports the idea of self-testing in self adaptive systems. In this method do not consider the notion of self-testable components. but, such components may be without difficulty included into this method, thereby enhancing the overall approach.

Denaro et al. [3] presents a method that automatically synthesizes assertions from the monitored behavior of the application aimed toward achieving adaptive application monitoring. The proposed method embeds assertions into the verbal exchange infrastructure of an utility that describes the legal interactions between the speaking entities. Those assertions are then checked at runtime to show misbehaviors, incompatibilities, and unexpected interactions which can occur because of hidden faults. The focus of the work is to provide systems with the capacity to automatically synthesize assertions that evolve over time and adapt to context-dependent interactions. The synthesis of assertions at runtime can benefit the self-testing of adaptive systems by offering a manner to generate extra test cases, which may be used to test components after a self adaptive change has been carried out.

Zhang et al. [17],[16] present an technique that formalizes the behavior of adaptive applications using state machines and Petri nets, respectively. The approach separates the model behavior from the non-adaptive behavior, making the models less complicated to specify and verify the usage of self-adaptive strategies.

IBM's architectural blueprint for Autonomic computing [8] defines a Common layered approach for describing self-managing systems. The 5 building blocks for the blueprint are touch points, autonomic managers, knowledge sources, a manual manager, and an enterprise service bus. Touchpoints implement force sensor and effector behavior for the manageability interfaces of managed resources[8], [10].

A managed resource is any type of hardware or software entity that can be controlled. Autonomic Managers may be either Touchpoint AMs or Orchestrating AMs, and may be organized in an hierarchical fashion [10]. Touchpoint AMs works at once with managed resources through their Touchpoints. Orchestrating AMs manage swimming pools of assets or optimize the Touchpoint AMs for

particular sources. The part of figure 1 to the left of the dashed line depicts an ordinary layered setup for AMs in an autonomic computing system. An Orchestrating AM is shown handling a single Touchpoint AM, which in turn manages a single hardware or software aid. Information assets are used to increase the built-in understanding capabilities of AMs. At the topmost level is the manual supervisor.

The contributions of the work by using Zhang and Chen [16] that may be implemented to this work includes: (1) specification of global invariants of the properties of adaptive applications regardless of adaptation and (2) introduction of state-based models that resource inside the technology of rapid prototypes. Using the specifications for global invariants and state-based models, test cases can be dynamically generated to test adapted software at runtime.

III. ADAPTIVE SYSTEMS AND TESTING EVOLVING

The evolving nature of the software program approach that systems normally need corrective, adaptive, or perfective maintenance [13] after initial deployment. Regression testing determines whether or not modifications to software have introduced new errors into previously tested code. This can contain re-running the whole test suite, or choosing a subset of the initial test suite for execution, i.e., selective regression testing [7]. Testing dynamically adaptive systems is extraordinarily tough due to the fact each the structure and behavior of the system may additionally change during execution. Existing test cases may additionally now not be applicable due to modifications in software structure, thereby requiring the creation of new test cases. Safe adaptation model maintained during adaptation [17], and is comprised of the three levels: analysis, detection and setup, and prepare a data structure for containing information such as component configurations, dependency relationships, and adaptive actions. The detection and setup levels takes place at runtime and entails generating safe adaption paths for acting adaptive actions on system components. The actual adaptation is then performed through the following steps:

1. Move the system into a partial operation mode in which some functionality of the component(s) to be adapted is disabled.
2. Hold the system in a safe state while adaptive actions are carried out.
3. Resume the system procedure' partial operation once all adaptive actions are complete.
4. Carry out a local-post action to go back the system to a completely-operational running state.

IV. OVERVIEW OF SELF-TESTING APPROACH

This approach contains testing activities into self-adaptive systems by imparting validation services to

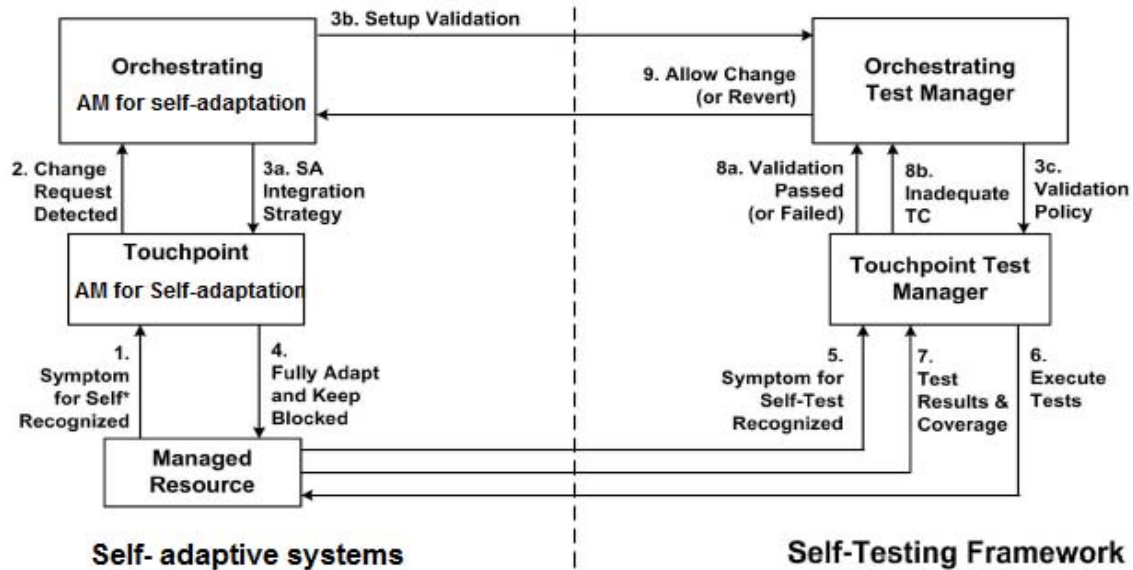


Fig 1: A Run time test model for self- adaptive systems

self- adaptive managers (AMs) through test interfaces. Figure 1 shows run time test model for self-adaptive systems. The left hand part of figure 1 indicates a minimal self-adaptive system and the right hand part indicates self-testing framework. The self-testing framework introduces test managers (TMs) that interact with AMs to dynamically validate exchange requests. This testing methodology can be divided into well known strategies which might be realization. At some stage in analysis, developers Primarily based on system overhead cost and guarantees that the integrity of system components is feasibility of use. Administrators can specify which method to apply for individual resources depending on factors such as the character of the resource (e.g., hardware or software); system configuration; and time and space requirements. On this phase we introduce test managers and outline their obligations, and provide exact steps of testing approach based on the two strategies - safe adaptation with validation, and replication with validation. Describe the workflow of Figure 1 when using the aforementioned strategies in Subsections 4.2 and 4.3 respectively.

A. Test Managers

Test managers (TMs) enlarge the concept of autonomic managers to testing activities, like AMs, TMs may also be orchestrating or Touchpoint [8]. Orchestrating TMs will coordinate excessive-level testing activities and manipulate Touchpoint TMs, while Touchpoint TMs performs low-level testing activities on managed resources. Here propose to integrate TMs inside self-adaptive systems to dynamically validate the alternate requests generated through AMs. TMs might be responsible for:

- Perform regression testing to make test out comes;
- Evaluating test outcomes and test coverage with respect to high-level validation policy;
- Preserving a test repository to allow storage, organization and access to test cases, test logs, and validation Policies.

To carry out their responsibilities, TMs will comprise components that enforce closed loops which might be consistent with the MAPE structure [8] of AMs.

B. Safe adaptation through validation

The safe adaptation with validation method validates adjustments because of self-control as part of a safe adaptation process [17], and takes place directly on the managed resources. This strategy may be used when it is too costly, impractical, or impossible to duplicate managed resources. The overhead cost of this approach is described by the amount of time the component under test remains blocked at the same time as awaiting validation to finish. The steps of this method correspond to the workflow of figure 1 (beginning from the left hand portion) as follows:

- A Touchpoint AM gathers data from a managed resource and correlates it into a symptom that warrants self-adaptation.
- A change request is generated by using the Touchpoint AM and this event is detected thru the sensor of an orchestrating am.
- The orchestrating AM initiates safe adaptation (3a.) and concurrently requests that an orchestrating tm sets up validation

(3b.) of the re-source. The correct validation Policy (3c.) is then loaded into the knowledge Component of a Touchpoint TM.

4. The Touchpoint AM proceeds with safe adaptation until the useful resource is completely-adapted (i.e., up to step three of the secure version procedure outlined in subsection 2.three) however maintains the resource blocked until validation is performed.
5. The Touchpoint TM detects that the resource is in a completely-adapted safe state, and is therefore ready to be tested.
6. After preliminary analysis and making plans, the Touchpoint TM executes test cases on the managed resource.
7. Test outcomes and test cases are making as a group and place into a log file, which is analyzed through the Touchpoint TM with respect to the validation policy.
8. A message indicating whether validation passed or failed (8a.), or if there was inadequate test coverage (8b.), is then sent to the orchestrating TM.
9. The orchestrating AM is notified as to whether or not to simply accept or reject the change request. primarily based on the validation policy, the orchestrating TM may either report to reject the change due to (8b.), or try to beautify the test suite and hold validation to acquire greater test coverage.

C. Replication Through validation

The replication with validation strategy requires the system to create and/or keep copies of the managed resource for validation process. Change requests for managed resources are first carried out on copies and proven prior to execution on the actual managed resources. The use of this method, step (3a.) of figure 1 could notify the Touchpoint AM to apply replication with validation as opposed to secure adaptation with validation. This will then redirect step (4.) to implement the change request on a effectively available copy of the resource, and the Touchpoint TM could perform validation using the changed replica. This kind of approach is handiest viable while managed assets may be replicated, and has the disadvantage of highly high overhead costs with recognize to the era and maintenance of resupply copies. But, the replication with validation approach has the gain that validation can occur without halting the regular operation of the system for the complete time required to check the aspect. In addition, validation does not need to occur on the same node because the autonomic gadget, and consequently the computational and storage overhead can be shifted so that there is little or no affect on system performance.

V. TOUCH POINT TMS ARCHITECTURE

In this section present the architecture of Touchpoint Test Managers and define the functions of their architectural components. Also describe the interactions between these components, and provide a high-level algorithm for validating change requests

A. Functions of components

The architectural components of Touchpoint TMs observe MAPE features in the context of testing process, and are defined as follows:

- a) Test monitor - retrieves structural information the alternate applied within the resource under test, and collects test outcomes and facts on test coverage. prior to validation, the test monitor continuously polls the useful resource to be tested, and acknowledges when it's far in a state appropriate for validation to start.
- b) Test analyzer - performs evaluation the usage of the current test suite, previous shape of the useful resource, and the new shape of the useful resource to decide applicable regression test cases; and develops new test cases to enhance the current test suite. The test analyzer also evaluates the test results and test coverage provided in the test log.
- c) Test planner - creates a test plan for the resource under test, which includes the test suite to be executed, test schedule, and a submit-test evaluation of the test log.
- d) Test executer - applies test cases to the useful resource under test.
- e) Test Knowledge - stores validation polices that encompass the place of the resources to be examined, its previous structure, its current specification, a baseline test suite, and test coverage criteria. The test knowledge will also serve as a central repository for extra test cases, test results, and test histories.

B. Component Interactions

The interactions of the architectural components are described by the change validation algorithm.

Begin ChangeRequestforvalidation

1.0 Load Validation Policy

// First Intelligent Closed Control Loop

1.1 new structure Retrieved

1.2 test suite analysis Performed

1.3 Request for creation of test plan

// External Monitored Event

1.3.1 Create Test Plan

```

// Corresponding Event Actuated
1.3.2 Test plan Finalized
1.4 test plan Execution
1.5 test cases Running
//second intelligent closed control loop
2.1 test results Retrieved
// External monitored events
2.2.1a Tests passed or tests failed
2.2.1b Inadequate Coverage
//corresponding event actuated
2.2.2a Testing process stopped
2.2.2b test suite reanalyzed (go to 1.2)

End ChangeRequestforvalidation

```

Fig 2: Change validation algorithm

An Orchestrating TM first sends a validation policy for the resource to a Touchpoint TM, which loads it into the test knowledge component (1.0). The Touchpoint TM will automatically invoke the test monitor to retrieve the new structure of the resource (1.1) when it senses that the resource is ready to be validated. The test monitor then sends the information about the new structure to the test analyzer (1.2), which uses it in conjunction with the previous structure and a baseline test suite to prepare a new test suite. The test analyzer notifies the test planner that a new test suite is ready and requests that a test plan be created (1.3). At this point, the Orchestrating TM detects that a Test Plan-Created event (1.3.1) has occurred and set up an external code profiling tool. Once the code profiler has been setup to instrument the managed resource for test coverage, the test plan is finalized and control is returned to the (1.3.2). The test plan is then passed to the test executor component (1.4) which then starts running test cases on the managed resource (1.5), and initiates the second loop.

The second loop commences with the retrieval of the test results and test coverage information (2.1) from the managed resource. The test monitor correlates this information into a test log and passes it to the test analyzer (2.2). The test analyzer evaluates the test log against the validation policy and determines whether validation passed or failed (2.2.1a), or if the test suite was inadequate (2.2.1b) with respect to the test coverage criteria. The results of the evaluation are sent to the Orchestrating TM, which would either end the validation process (2.2.2a), or continue testing after reanalyzing the current test suite (2.2.2b). If a decision is made to reanalyze the test suite, the behavior of the second loop from 2.3 to 2.5 is the same as the first loop from 1.3 to 1.5.

VI. EXPERIMENTAL RESULTS

To validate this approach, develop a prototype of a java application that has self-adaptive capabilities. This prototype implements a data structure that has the ability to self-configure at runtime, which refer to

as an adaptive repository. the managed element of the adaptive repository become applied as a stack, and an outside utility that makes random requests to push (pop) data elements to (from) the stack was developed. When the stack is an average of 80 percentage of its complete capacity, the AM for self-configuration attempts to reconfigure the stack by increasing its potential. This reconfiguration is intercepted and a signal is sent the self-testing issue to validate the exchange request on a replica of the stack. Equipment has been used to help testing: JUnit [6] and Cobertura [4]. The high-level validation policy for the prototype required 100% pass rate for all test cases executed, and at least 75% branch and statement coverage. The initial test suite for the adaptive repository included 15 test cases developed using a combination of test strategies. These test strategies included: boundary, random, and equivalence partitioning [18]. Test cases to validate properties related to the capacity of the repository were parameterized. To evaluate the prototype for the testing framework, utilized a mutation testing technique to simulate two change request scenarios:

- 1) Request for correct reconfiguration of the stack,
 - 2) Request for incorrect reconfiguration of the stack.
- The latter was achieved by altering the method that reconfigures the stack capacity to cause a decrease in capacity rather than an increase. In the first scenario, all of the test cases passed and the code coverage criteria were met (75% branch, 80% statement). The second scenario produced two test case failures, and therefore code coverage was omitted. These results were favorable as validation failed for the incorrect reconfiguration scenario, and hence would have prevented a potentially harmful change request from being implemented on the stack.

Building the prototype provided us with insight on the scope of the self-test manager in terms of the operations that should be performed by the autonomic system and the self-testing framework. Dynamic test case generation and code-based changes remain core limitations of the prototype. Currently the prototype only implements replication with validation.

VII. CONCLUSION

In this paper proposed a framework that dynamically validates changes in self-adaptive systems. This framework extends the current structure of self-adaptive software architecture to consist of self-testing as an implicit characteristic. Proposed two techniques to support testing at runtime: safe adaptation with validation, and replication with validation. To show the feasibility of this testing framework developed a prototype that uses self adaptive systems.

REFERENCES

- [1] M. Blum, M. Luby, and R. Rubinfeld. Self-testing/correcting with applications to numerical problems. *Journal of Computer and System Sciences* ,45(6):

- 549595, 1993.
- [2] G. Denaro, L. Mariani, and M. Pezzè. Self-test components for highly reconfigurable systems. *Electronic Notes in Theoretical Computer Science*, 82(6), 2003.
- [3] G. Denaro, L. Mariani, M. Pezzè, and D. Tosi. Adaptive runtime verification for autonomic communication infrastructures. In *First International IEEE WoW-MoM Workshop on Autonomic Communications and Computing (ACC'05)*, pages 553–557, 2005.
- [4] M. Doliner, G. Lukasik, and J. Thomerson. Cobertura 1.8, 2002. <http://cobertura.sourceforge.net/> (August 2006).
- [5] Enterprise Management Associates. Practical autonomic computing: Roadmap to self managing technology. Technical report, IBM, Boulder, CO, Jan. 2006.
- [6] E. Gamma and K. Beck. JUnit 3.8.1, 2005. <http://www.junit.org/index.htm> (August 2006).
- [7] T. L. Graves, M. J. Harrold, J.-M. Kim, A. Porter, and G. Rothermel. An empirical study of regression test selection techniques. *ACM Trans. Softw. Eng. Methodol.*, 10(2):184–208, 2001.
- [8] IBM Autonomic Computing Architecture Team. An architectural blueprint for autonomic computing. Technical report, IBM, Hawthorne, NY, June 2006.
- [9] IBM Corporation. IBM Research, 2002. <http://www.research.ibm.com/autonomic/> (August 2006).
- [10] J. Kephart and D. Chess. The vision of autonomic computing. *Computer*, 36(1):41–52, January 2003.
- [11] R. Kumar and D. Sivakumar. On self-testing without the generator bottleneck. In *Proceedings of the 15th Conference on Foundations of Software Technology and Theoretical Computer Science*, pages 248–262, London, UK, 1995. Springer-Verlag.
- [12] H. A. Muller, L. O'Brien, M. Klein, and B. Wood. Autonomic computing. Technical report, Carnegie Mellon University and Software Engineering Institute, April 2006.
- [13] I. Sommerville. *Software Engineering: Seventh Edition*. Addison-Wesley, Essex, England, 2004.
- [14] Y. L. Traon, D. Deveaux, and J.-M. Jézéquel. Self-testable components: From pragmatic tests to design-for-testability methodology. In *Technology of Object-Oriented Languages and Systems (TOOLS 99)*, pages 96–107. IEEE Computer Society, 1999.
- [15] H. Wasserman and M. Blum. Software reliability via runtime result-checking. *J. ACM*, 44(6):826–849, 1997.
- [16] J. Zhang and B. H. C. Cheng. Model-based development of dynamically adaptive software. In *ICSE '06: Proceeding of the 28th international conference on Software engineering*, pages 371–380, New York, NY, USA, 2006. ACM Press.
- [17] J. Zhang, B. H. C. Cheng, Z. Yang, and P. K. McKinley. Enabling safe dynamic component-based software adaptation. In *WADS*, pages 194–211, 2004.
- [18] H. Zhu, P. A. V. Hall, and J. H. R. May. Software unit testing coverage and adequacy. *ACM Computing Surveys*, 29(4):366–427, December 1997.